

Note 3: Jobs, Steps, Actions & Runners in Detail

Jobs

Defining a Job

Each job has a unique ID, specifies which runner to use, and contains a list of steps.

jobs:

```
build:                # Job ID

  name: Build and Test  # Display name (optional)

  runs-on: ubuntu-latest  # Runner specification

  steps:

    - ...
```

Parallel Jobs (Default Behaviour)

By default, all jobs in a workflow run in parallel on separate runners. This is useful when you have independent tasks like running tests on different operating systems simultaneously.

jobs:

```
test-linux:

  runs-on: ubuntu-latest

  steps:

    - run: echo "Testing on Linux"
```

```
test-windows:

  runs-on: windows-latest

  steps:

    - run: echo "Testing on Windows"
```

Both test-linux and test-windows start at the same time on separate virtual machines.

Sequential Jobs Using needs

When one job depends on another completing successfully first, use the needs keyword:

jobs:

```
build:
```

runs-on: ubuntu-latest

steps:

- run: echo "Building..."

test:

runs-on: ubuntu-latest

needs: build # Only runs after build succeeds

steps:

- run: echo "Testing..."

deploy:

runs-on: ubuntu-latest

needs: [build, test] # Only runs after both build AND test succeed

steps:

- run: echo "Deploying..."

If build fails, test and deploy are skipped automatically.

Job Outputs

Jobs can pass data to subsequent jobs using outputs:

jobs:

build:

runs-on: ubuntu-latest

outputs:

version: \${ steps.get-version.outputs.version }

steps:

- id: get-version

- run: echo "version=1.0.0" >> \$GITHUB_OUTPUT

deploy:

needs: build

runs-on: ubuntu-latest

steps:

- run: echo "Deploying version \${needs.build.outputs.version}"

Steps

Steps are the individual tasks within a job. They run sequentially on the same runner. Each step either runs a shell command using run or uses a pre-built action using uses.

Step with run (Shell Command)

steps:

- name: Print Java version

run: java -version

- name: Run Maven build

run: mvn clean package -DskipTests

- name: Multi-line shell script

run: |

echo "Starting build..."

mvn clean package

echo "Build complete."

The | character in YAML allows multi-line strings. Each line is a separate shell command.

Step with uses (Pre-built Action)

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Set up Java 17

```
uses: actions/setup-java@v4
```

```
with:
```

```
  java-version: '17'
```

```
  distribution: 'temurin'
```

The uses keyword references an action. The format is owner/repo@version.

- actions/checkout@v4 — official GitHub action at version 4
- with — passes input parameters to the action

Step Conditions

Steps can be conditionally executed using if:

```
steps:
```

```
- name: Deploy to production
```

```
  if: github.ref == 'refs/heads/main'
```

```
  run: ./deploy.sh
```

```
- name: Run only on failure
```

```
  if: failure()
```

```
  run: echo "Something went wrong!"
```

Common condition functions:

- success() — runs if all previous steps succeeded (default)
- failure() — runs if any previous step failed
- always() — runs regardless of previous step results
- cancelled() — runs if the workflow was cancelled

Environment Variables in Steps

```
steps:
```

```
- name: Build with env var
```

```
  run: echo "Building $APP_NAME version $APP_VERSION"
```

```
  env:
```

```
    APP_NAME: my-app
```

APP_VERSION: 1.0.0

Actions

What is an Action?

An action is a reusable, self-contained piece of automation. When you use `uses:` in a step, you are invoking an action. Actions can be:

- JavaScript actions — run directly on the runner
- Docker container actions — run inside a Docker container
- Composite actions — combine multiple steps into one reusable action

Commonly Used Official Actions

`actions/checkout` Checks out your repository code onto the runner so subsequent steps can access it. This is almost always the first step in any job.

- `uses: actions/checkout@v4`

with:

`fetch-depth: 0` # Fetch full history (needed for some tools)

`actions/setup-java` Installs a specific Java version on the runner.

- `uses: actions/setup-java@v4`

with:

`java-version: '17'`

`distribution: 'temurin'` # Options: temurin, zulu, corretto, microsoft

`cache: 'maven'` # Automatically caches Maven dependencies

`actions/setup-node` Installs a specific Node.js version.

- `uses: actions/setup-node@v4`

with:

`node-version: '20'`

`cache: 'npm'`

`actions/cache` Caches files between workflow runs to speed up builds.

- `uses: actions/cache@v4`

with:

path: ~/.m2/repository

key: \${{ runner.os }}-maven-\${{ hashFiles('**/pom.xml') }}

restore-keys: |

\${{ runner.os }}-maven-

actions/upload-artifact and actions/download-artifact Share files between jobs by uploading them as artifacts.

In build job

- uses: actions/upload-artifact@v4

with:

name: jar-file

path: target/*.jar

In deploy job

- uses: actions/download-artifact@v4

with:

name: jar-file

Runners

GitHub-Hosted Runners

GitHub provides virtual machines that are automatically provisioned for each job and discarded after the job completes. They come pre-installed with common tools.

Runner Label	Operating System	Common Pre-installed Tools
ubuntu-latest	Ubuntu 22.04	Java, Node.js, Python, Docker, Maven, Git
windows-latest	Windows Server 2022	Java, Node.js, Python, .NET, Git
macos-latest	macOS 13	Java, Node.js, Python, Xcode, Git

jobs:

build:

runs-on: ubuntu-latest

GitHub-hosted runners are the default choice for most projects. They are clean, isolated, and require no maintenance.

Self-Hosted Runners

A self-hosted runner is a machine you own and manage that is registered with GitHub to run workflow jobs. You install the GitHub Actions runner application on your own server or VM.

jobs:

build:

runs-on: self-hosted

Or with labels for more specific targeting:

jobs:

build:

runs-on: [self-hosted, linux, x64, gpu]

When to use self-hosted runners:

- Your build requires access to internal network resources (private databases, internal APIs)
- You need specific hardware (GPUs, ARM processors, high memory)
- You want to avoid GitHub's usage limits and billing
- Your organisation has compliance requirements about where code is executed
- Your builds are very frequent and self-hosting is more cost-effective

Runner Security Considerations:

- Never use self-hosted runners for public repositories — a malicious pull request could execute arbitrary code on your runner
- Use separate runners for different sensitivity levels (test runners vs production deployment runners)
- Regularly update the runner application
- Run the runner as a low-privilege user, not as root
- Use ephemeral runners (runners that are destroyed after each job) for better security isolation

Putting It All Together — A Complete Basic CI Workflow

name: Java CI Pipeline

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

build-and-test:

name: Build and Test

runs-on: ubuntu-latest

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Set up Java 17

uses: actions/setup-java@v4

with:

java-version: '17'

distribution: 'temurin'

cache: 'maven'

- name: Build with Maven

run: mvn clean package

- name: Upload JAR artifact

uses: actions/upload-artifact@v4

with:

name: app-jar

path: target/*.jar

retention-days: 7

This workflow:

- Triggers on pushes to main or develop, and on pull requests targeting main
 - Runs on a GitHub-hosted Ubuntu runner
 - Checks out the code
 - Sets up Java 17 with Maven dependency caching
 - Runs a full Maven build including tests
 - Uploads the resulting JAR as a downloadable artifact retained for 7 days
-

.